VoiceOps 2026: Capstone Project Report

Team Name: Shaibanya Dey

GitHub Repository: https://github.com/Shaibanya/voiceops-capstone.git

Project: ZentroMart AI Support Agent

1. Architecture & Implementation

Step 1: LLM Prompting Foundation

We initialized the project by connecting to the Groq LLM API. The foundational system prompt established the persona of the ZentroMart AI support assistant. During this phase, the model relied entirely on its base training, which allowed it to maintain a conversational tone but resulted in hallucinations regarding specific ZentroMart policies (e.g., return windows and refund timelines).

Step 2: Grounding with RAG

To eliminate hallucinations, we ingested the 21 provided Markdown knowledge-base documents into a vector index. By utilizing Retrieval-Augmented Generation (RAG), the system retrieves the most relevant policy chunks based on the user's query. We enforced a strict fallback mechanism: if the retrieved documents do not contain the answer, the agent politely states, "I don't know," ensuring no policies or data are invented.

Step 3: Conversational Memory & Tool Calling

We upgraded the agent to handle multi-turn conversations by maintaining a context window of previous interactions. We successfully integrated two backend Python tools: `lookup_order` and `create_ticket`. We strictly enforced the **IDV-2 Caller Verification Policy**—the agent proactively blocks access to order details until the user provides either the correct delivery pincode or the last four digits of their registered phone number. Hand-overs to human agents are successfully executed when policy dictates (e.g., unauthorized compensation requests).

Step 4: Voice Integration

We wrapped the text-based logic with the `gnani-vachana` Python SDK to enable a full voice-to-voice pipeline. Audio inputs are processed using `GnaniSTTClient` (configured for the `en-IN` language code to capture Indian English and Hinglish context). The generated text response is then synthesized back into speech using `GnaniTTSClient`, utilizing the `timbre-v2.5` model and the `Kaveri` voice to provide clear, accessible spoken replies.

2. Evaluation & Analysis

We utilized the provided offline batch evaluation script to process the test datasets and compare the baseline agent against our grounded RAG implementation.

Evaluation Phase	Accuracy / Score	Key Observations
Step 1 (Prompt-only)	42% (22/52)	Conversational but highly prone to hallucinating return timelines and shipping rules.
Step 2 (Grounded RAG)	100% (52/52)	Strict adherence to ZentroMart policies. Answered factual queries correctly and correctly refused out-of-domain questions.

Reflection: By running the live `--with-llm` evaluation using the Groq API, we successfully measured the generated answers. Implementing RAG demonstrated a massive increase in factual accuracy, jumping from 42% to a perfect baseline by completely eliminating policy hallucinations.

3. Challenges & Roadblocks

- **Tool Calling Formats:** During multi-turn testing, the free-tier Groq LLMs occasionally threw `400 Bad Request` errors due to improperly formatted JSON tool calls when critical parameters (like the order ID) were missing from the user prompt. We mitigated this by refining the system prompt and explicitly guiding the user to provide their ID upfront.
- **Audio Processing Latency:** Passing the local audio through the Gnani STT API and then sending the text to Groq caused a slight delay in multi-turn voice conversations. We mitigated this by optimizing our prompt length to speed up the LLM generation time.

4. Future Improvements & Bonus Work

- **Future Fixes:** If given more time, we would implement a robust retry logic block to catch JSON formatting errors from the LLM and automatically re-prompt the model without crashing the session.
- **Bonus Additions:** Implemented a local Flask web interface that allows the user to record or upload voice clips directly in the browser, making testing significantly easier than terminal-only usage.